# WEB-BASED ML MODEL EVALUATOR

## Branch Coverage Report

| | |
|---|---|
| Project | Web-Based ML Model Evaluator (Team SE-Jackfruit) |
| Backlog item | BL-15 - Code Coverage and Branch Coverage Reports |
| Result | Branch coverage 88.9% - 240 of 270 decision outcomes taken |

## 1. Purpose

Branch coverage is stricter than line coverage. Wherever the program makes a decision (an 'if', a loop, an error handler), there are at least two ways to go. A branch counts as covered when the tests made the program take that way. A line can be run while the other outcome of its decision is never tried; branch coverage finds exactly those cases.

Example: for `if key exists: use it`, line coverage is satisfied by a test where the key exists. Branch coverage also needs a test where it does not.

The numbers come from coverage.py, the standard Python coverage tool, driven by pytest-cov while the same 219 tests of BL-13 ran. No test was added or changed for this report.

## 2. Method and environment

| Item | Detail |
|---|---|
| Test run | 5 November 2026, 16:46 IST (timestamp in the coverage file) |
| Environment | Windows, Python 3.14.6, pytest 9.1.1, pytest-cov 7.1.0, coverage.py 7.17.2 |
| Command | python -m pytest tests -q -p no:cacheprovider --cov=app --cov=config --cov=services --cov-branch --cov-report=term-missing --cov-report=json --cov-report=xml --cov-report=html |
| Tests run | The full suite: 219 tests, all passed (63.1 s) |
| Code measured | The application code only: app.py, config.py and the services folder (15 files, 1,110 statements, 270 branches). The test files themselves are not measured. |
| Evidence files | coverage-output.txt, coverage.json and coverage.xml in docs/test-results/coverage |

## 3. Overall result

| Decision outcomes (branches) | Taken by tests | Not taken | Decisions only half-tested | Branch coverage |
|---|---|---|---|---|
| 270 | 240 | 30 | 22 | 88.9% |

The code contains 270 decision outcomes. The tests took 240 of them (88.9%). 30 outcomes were never taken; they belong to 22 decisions where one outcome was tested and the other was not. For comparison, line coverage of the same run is 95.8% (see the Code Coverage Report). Branch coverage is lower, as expected, because it asks for more.

## 4. Branch coverage by file

| File | Branches | Taken | Not taken | Branch coverage |
|---|---|---|---|---|
| app.py | 50 | 45 | 5 | 90.0% |
| config.py | 6 | 4 | 2 | 66.7% |
| services/__init__.py | 0 | 0 | 0 | n/a |
| services/admin_service.py | 22 | 21 | 1 | 95.5% |
| services/auth_service.py | 28 | 25 | 3 | 89.3% |
| services/compatibility.py | 6 | 5 | 1 | 83.3% |
| services/dataset_service.py | 26 | 24 | 2 | 92.3% |
| services/db.py | 4 | 4 | 0 | 100.0% |
| services/errors.py | 0 | 0 | 0 | n/a |
| services/evaluation_service.py | 42 | 40 | 2 | 95.2% |
| services/metrics.py | 4 | 4 | 0 | 100.0% |
| services/model_service.py | 18 | 17 | 1 | 94.4% |
| services/report_service.py | 20 | 16 | 4 | 80.0% |
| services/timeutil.py | 0 | 0 | 0 | n/a |
| services/visualization_service.py | 44 | 35 | 9 | 79.5% |
| Total | 270 | 240 | 30 | 88.9% |

Files with no decisions (errors.py, timeutil.py, the package marker) show no branches.

## 5. Decision outcomes the tests did not take

'From to' gives the line holding the decision and the line the program would go to. Each row is one outcome that no test took.

| File | Decision at line | Goes to line | Meaning |
|---|---|---|---|
| app.py | 123 | 125 | Browser (non-API) request hits an unexpected error: generic error page. |
| app.py | 140 | 141 | Home address '/' opened while logged out: redirect to login. |
| app.py | 140 | 142 | Home address '/' opened while logged in: redirect to the user's home page. |
| app.py | 166 | 167 | Register page opened while already logged in. |
| app.py | 501 | 502 | File run directly instead of through 'flask run'. |
| config.py | 40 | 41 | Secret key supplied by environment variable. |
| config.py | 43 | 44 | Secret-key file does not exist yet (first start). |
| services/admin_service.py | 31 | 32 | Caller is not an active Admin (blocked earlier by the routes). |
| services/auth_service.py | 28 | 29 | Public view of a missing user. |
| services/auth_service.py | 41 | 42 | Name longer than 80 characters. |
| services/auth_service.py | 62 | 63 | Role other than analyst or admin. |
| services/compatibility.py | 22 | 27 | Model states neither feature names nor feature count. |
| services/dataset_service.py | 110 | 111 | Target column missing at validation time. |
| services/dataset_service.py | 167 | 169 | Deleting a dataset whose stored file is already gone. |
| services/evaluation_service.py | 99 | 102 | ROC-AUC cleared after the probability function failed. |
| services/evaluation_service.py | 144 | 145 | Public view of a missing evaluation. |
| services/model_service.py | 134 | 136 | Deleting a model whose stored file is already gone. |
| services/report_service.py | 45 | 46 | Metric value missing in a report. |
| services/report_service.py | 131 | 132 | Report notes: the case with no notes. |
| services/report_service.py | 133 | 134 | Report notes loop: entered case. |
| services/report_service.py | 133 | 135 | Report notes loop: finished case. |
| services/visualization_service.py | 53 | 55 | Chart value that is not a plain number or text. |
| services/visualization_service.py | 55 | 56 | Numpy decimal value converted for the chart data. |
| services/visualization_service.py | 55 | 57 | Value that is not a numpy decimal. |
| services/visualization_service.py | 57 | 58 | Numpy true/false value converted. |
| services/visualization_service.py | 57 | 59 | Value that is neither numpy decimal nor true/false. |
| services/visualization_service.py | 65 | 72 | More classes than the confusion matrix limit: matrix is left out. |
| services/visualization_service.py | 79 | 80 | Class missing from the data: ROC curve skipped. |
| services/visualization_service.py | 88 | 90 | No ROC curve could be drawn for any class. |
| services/visualization_service.py | 162 | 168 | More than 15 classes: numbers are not written inside the confusion matrix cells. |

## 6. Assessment

**Where branch coverage is lowest.** config.py (66.7 %), visualization_service.py (79.5 %) and report_service.py (80.0 %). In config.py both gaps are the secret-key set-up. In visualization_service.py most gaps are number-type conversions and chart limits for very many classes. In report_service.py they are the optional 'Notes' section of a report.

**What is well covered.** The rules that protect users and data are tested in both directions: login, roles, last-admin protection, per-user isolation, file validation and compatibility checks. admin_service.py (95.5 %), model_service.py (94.4 %) and evaluation_service.py (95.2 %) are all high.

**Nature of the untested outcomes.** They fall into four groups: (1) start-up and run-directly code, (2) damaged-file and failed-write situations, (3) rarely used chart and report options, and (4) a second check of a rule that is already enforced earlier in the program. None is on a main user journey.

**Why the gaps were left.** The tests were not changed for this report, so the results of BL-13 and BL-14 stay valid (219 tests, 86 of 86 cases). Adding tests only to raise a percentage was judged less valuable than reporting the figures honestly.

## 7. Conclusion

Branch coverage is 88.9% (240 of 270). The tests exercise both outcomes of almost every decision on the main paths. The outcomes that remain are listed with their meaning in section 5 and are candidates for further tests in a later iteration, starting with the secret-key set-up and the report notes section.